Recommender Systems - Course Project Report

Cumulative report template - Parts 1, 2, and 3

Team	Group 23
Members	Van Bui Nguyen Duc Minh Tiina Korkolainen
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/tiinakor/recommender-systems-group-23

How to use this template

- *This is one cumulative report. Extend and revise the same document at each project stage.*
- *Maximum length: 4 pages at Part 1, 8 pages at Part 2, and 12 pages for the final report, excluding references.*
- *Keep the section headings below. You may add subsections where useful.*
- *All explanations and justifications requested in the project specification belong in this report.*
- *The Results Sheet contains the complete standardized numerical record. In the report, include only the tables and figures needed to support your analysis.*
- *Delete all italic instructional text before the final submission.*

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

MovieLens 1M contains 1,000,209 ratings on a scale of 1 to 5 stars from 6,040 users. Of the 3,883 movies listed, 3,706 have at least one rating. The density over users and rated movies is 4.47%, far above the 1% or less described in Lecture 1.1. No user rates the same movie twice, and every user has at least 20 ratings, so GroupLens has already filtered the data on user activity.

Ratings are concentrated on a small part of the catalogue (Figure 1). The 370 most-rated movies, 10% of those rated, receive 44.4% of all ratings, while 446 movies have fewer than 10. Users are less skewed, with a median of 96 ratings and a maximum of 2,314. We therefore expect a most-popular baseline to be strong, and item-based kNN to have little co-rating evidence for tail movies.

Ratings lean positive. Four stars is the most common rating (34.9%), and ratings of 4 or 5 make up 57.5% of the data, which Section 1.2 treats as positive feedback.

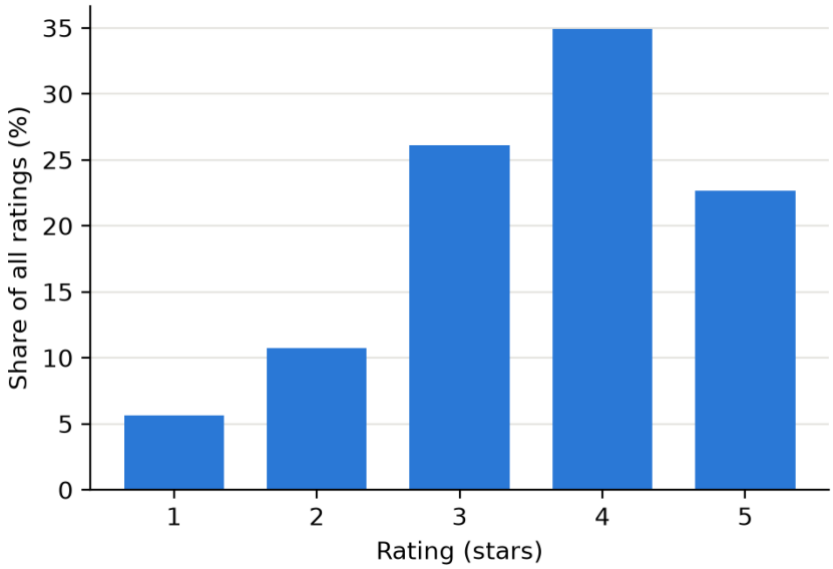


Figure1. Ratings per movie and per user

The ratings span April 2000 to February 2003, but their distribution over time is very uneven (Figure 2): 90.5% were entered in 2000, with a peak in November. Individual users are just as bursty. Half of all users entered every rating within 68 minutes, and 62.9% within a single day, and since timestamps record when a rating was entered, a single session can cover films watched years apart. This burstiness shapes the temporal split, because many users who rate after a cut have no earlier ratings (Section 1.3).

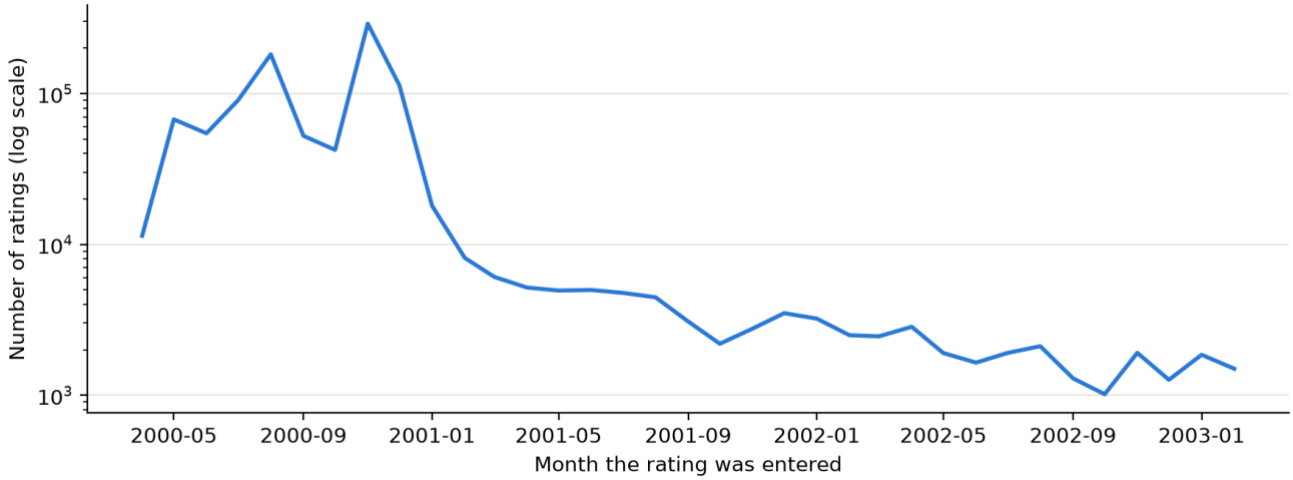


Figure 2. Number of ratings entered per month

We treat the user attributes, gender and seven age bands, only as rough proxies: they are self-reported, gender is recorded as binary, and the data dates from 2003.

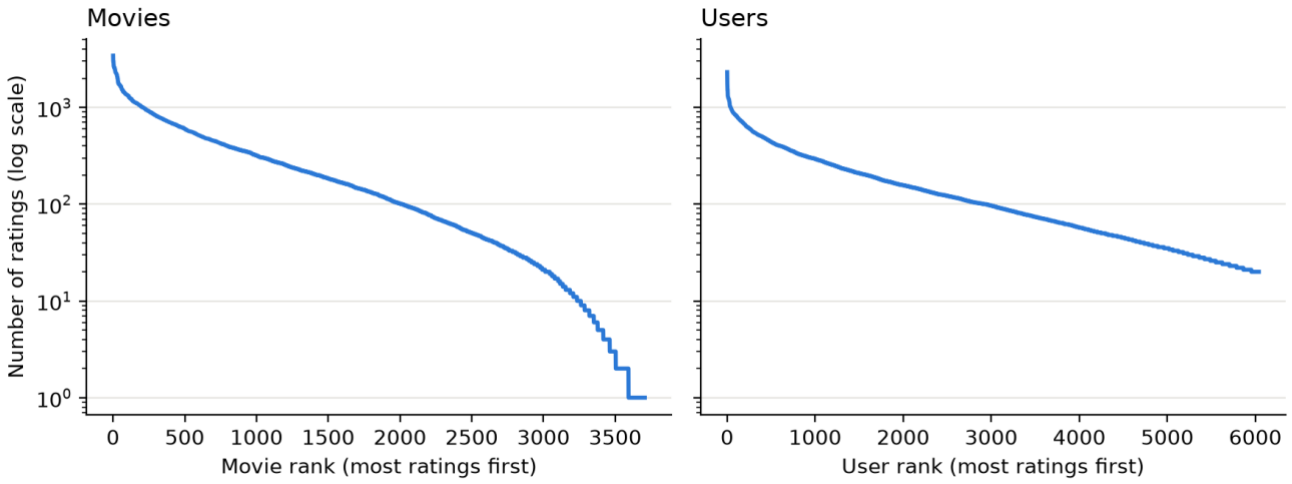


Figure 3. Number of ratings based on movie rank and user rank

1.2 Preprocessing and feedback treatment

We apply no extra activity filtering. GroupLens already guarntees at least 20 ratings per user, and dropping rare movies would hide the long tail that Section 1.1 describes. We treat a rating of 4 or 5 as positive feedback, and lower ratings as not relevant. This keeps 57.5% of all ratings as positives, and it avoids rewarding a model for recommending films a user disliked. All ratings, positive or not, are used as training input, and every movie a user rated in training is excluded from that user's recommendations.

1.3 Train/validation/test split

We sort all 1,000,209 ratings by timestamp and cut the global timeline at 70% and 85% of ratings. Training covers 25 Apr – 22 Nov 2000 03:06 (700,148 ratings), validation runs to 10 Dec 2000 04:20 (150,029 ratings), and test covers everything after, up to 28 Feb 2003. Ratings with identical timestamps always fall on the same side of a cut, so the split is deterministic. A global cut means no model is trained on ratings entered after the ones it predicts.

Because ratings arrive in bursts (Section 1.1), many users have no ratings before a cut, and no collaborative model can score a user without history. Of the 1,352 users who rate during the validation period, 845 are new, so when tuning we score only the remaining 507 users (35,138 of the 150,029 validation ratings). For the final run, models are refit on training + validation, so a test user only needs history in either set. Of 1,512 test users, 325 are new, leaving 1,187 users and 101,667 ratings. Hyperparameters are therefore tuned on about 3.5% of all ratings, and validation scores are correspondingly noisy.

	All users	**New (no history)**	**Scored**
Validation (history = train)	1,352	845	**507**
Test (history = train + validation)	1,512	325	**1,187**

1.4 Evaluation protocol

For every user we rank all 3,706 rated movies, remove the ones the user rated in training, and take top 10. We do not sample negatives, because sampled metrics can reverse the ordering of the models. We report

Precision@10, Recall@10 and NDGC@10, with binary gains discounted by 1 / $\log_2$(rank+1) and an ideal list of min(10, |relevant| hits; k = 10 matches a ten-item interface. Catalogue coverage is the share of the 3,706 movies that appear in at least one user's top 10. Metrics are averaged per user with equal weight, over users with at least one relevant held-out movie: 475 on validation and 1,161 on test. Hyperparameters are selected by validation NDCG@10 only, and test is used once, after refitting on training + validation.

1.5 Baseline methods and settings

Our item-based kNN follows Lecture 1.2 (slides 5 to 10). We compute the similarity between every pair of movies once from the training matrix. To score a candidate movie for a user, we take the k movies the user has rated that are most similar to it, discard negative similarities, and combine the user's ratings on these neighbours, either as a similarity-weighted average or as an unnormalised sum Σ sim · r. Movies the user has already rated are excluded from the recommendations.

We tuned the model on the 475 validation users with training history, comparing 30 configurations by NDCG@10: cosine, adjusted cosine or Jaccard similarity, k ∈ {10, 20, 50, 100, 200}, and both aggregation rules. The aggregation rule had by far the largest effect. Summed configurations scored between 0.149 and 0.181, while the best averaged configuration reached only 0.040. Under averaging, a movie with a single weak neighbour that the user rated 5 receives the maximum score, so obscure movies rise to the top of the list. We therefore use summed scores with cosine similarity and k = 20, which gave the highest validation NDCG@10 (0.181). Cosine is the lecture's default for item-item similarity, and k = 20 lies within the typical range of 20 to 50. Cosine with k = 10 or k = 100 scored within about 0.001 of this configuration, a difference too small to distinguish on 475 users.

Regarding most popular, this baseline recommends the most frequently rated training movies, skipping any the user has already rated. Since it has no hyperparameters, we report all four variants, which count either all ratings or only positive ones (4 or 5 stars), over either the whole training period or its last 120 days. The 120-day window was fixed in advance and not tuned.

1.6 Results and analysis

Item-based kNN scores highest on every accuracy metric for the 1,161 test users, with Precision@10 = 0.210, Recall@10 = 0.058 and NDCG@10 = 0.229. The best Most Popular variant, which counts only positive ratings over the whole training period, reaches 0.202, 0.055 and 0.219 (the full results for all variants are in the Results Sheet). The lead of item-kNN is small, however: 0.010 NDCG@10 on test, while on validation the two models were nearly tied (0.181 against 0.180). Without confidence intervals we cannot claim that item-kNN is more accurate, and we will test this difference in Part 2.

Coverage separates the two models more clearly. Item-kNN recommends 8.5% of the 3,706 movies to at least one user, compared with 3.8% for the best Most Popular variant. This is expected, since Most Popular gives almost every user the same list. Even so, item-kNN still reaches only a small part of the catalogue, in line with the long tail in Section 1.1: rarely rated movies seldom appear among anyone's nearest neighbours.

Within Most Popular, counting only positive ratings raises NDCG@10 from 0.206 to 0.219. Restricting the count to the last 120 days of training changes NDCG@10 by at most 0.001 for either counting rule, which is unsurprising given that 90.5% of all ratings were entered in 2000 and the most popular movies changed little over that period.

All models score higher on test than on validation (0.229 against 0.181 for item-kNN). Two factors may contribute: the final models are trained on more data, and the validation and test periods contain different

groups of users (Section 1.3). We have not yet tested either explanation. Recall@10 is low for every model because many test users have more relevant movies than fit in a list of ten.

2. Part 2 - Models and Honest Comparison

2.1 Models

Describe the additional models and paradigms. Clearly identify the self-implemented algorithm and explain its implementation.

2.2 Hyperparameter tuning and variability

Describe the validation-only tuning procedure, search spaces/budgets, random seeds, and confidence intervals or statistical testing.

2.3 Overall results

Present and discuss the main comparison. Explain important differences and avoid unsupported claims.

2.4 Disaggregated analysis

Report and interpret results broken down by at least two of the required dimensions: items with few vs. many training interactions, low- vs. high-activity users, popular vs. long-tail items, and early vs. late test period. Define any thresholds used and focus on findings that change or qualify the overall conclusions.

2.5 Sparsity ablation

Report and discuss the 100%, 50%, and 20% training-interaction experiments, including changes in the ranking between models.

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

State Track A, B, or C and briefly motivate the selected intervention/extension.

3.2 Method

Describe only your selected track. The Track A/B/C guidance below is provided as instructions; delete the guidance for the two tracks you did not select before submission.

If Track A - Diversity and popularity bias

State the base recommender, selected re-ranking method, genre-vector item representation, trade-off parameter, and exact item-distance measure. Describe how the parameter was swept.

If Track B - Fairness

Define the release-decade provider proxy, report and justify the final decade boundaries and group sizes, state and justify the adopted fairness notion, describe the consumer groups based on MovieLens user attributes, and identify the stage and form of the mitigation.

If Track C - Group recommendations

State the selected synthetic group type and justify the chosen group size (3-5). Describe the three aggregation strategies. Identify which strategy was extended to the sequential scenario and explain how satisfaction is carried forward between recommendation rounds.

3.3 Results and trade-offs

Present the required track-specific results. Track A: report intra-list diversity, catalogue coverage, Gini and calibration error across the parameter sweep, and discuss the accuracy-diversity frontier. Track B: report position-weighted exposure against catalogue share, per-group consumer-side accuracy, and the fairness-performance frontier. Track C: compare group satisfaction, disagreement and minimum member satisfaction across the three aggregation strategies, then analyse how member and group satisfaction develop in the sequential scenario.

3.4 Cost and interpretation

State what is gained, what is lost, and who bears the cost where applicable. Explain what operating choice or strategy you would select and why.

3.5 "One number we cannot explain"

Identify one result you do not fully understand and describe how you would investigate it.

4. Discussion and Conclusions

Summarize the main findings across the project. Discuss limitations, failed approaches where relevant, and what the experiments support.

5. Contribution Statement

For each team member, provide two or three sentences describing their contribution.

6. Use of AI Assistants

State which AI tools were used and for what. If none were used, state that explicitly.

References

List references used in the report. References do not count toward the page limit.